**DEPARTMENT OF ELECTRICAL ENGINEERING**

**College of Electrical and Mechanical Engineering (CEME), NUST-Pakistan**

**B.E. Electrical Engineering**

## EE-330 Digital Signal Processing

# Project Report

**Date: _____**

| Group Members | Registration Number | Syndicate |
|---|---|---|
| Abdul Haseeb Riaz | 429083 | B |
| Aoun Muhammad | 409738 | |
| Muhammad Zain-ul-Abideen | 407102 | |
| | | |

| Software Usage (P3) | Debugging and Results (P3) | Individual and Teamwork (A3) | Lab Report (A3) | Total |
|---|---|---|---|---|
| | | | | |

# EEG Signal Filtering and Real-Time Visualization – Lab Report

## a. Comprehensive Report

This project focuses on preprocessing EEG signals using MATLAB to clean and visualize brainwave data collected during mental arithmetic tasks. The process includes:

1. Loading EEG signals from CSV files.
2. Applying digital filters (low-pass, high-pass, notch) to remove noise and artifacts.
3. Simulating real-time EEG data visualization using sliding window plots.

The outcome is a cleaner EEG signal, free of high-frequency noise, DC drifts, and power line interference, along with a smooth real-time simulation mimicking a live system.

## b. MATLAB Installation Summary

- **Software:** MATLAB R2018b
- **Toolboxes Used:**
  - Signal Processing Toolbox (for filtering functions)
  - MATLAB Base Environment

**Installation Steps:**

1. Download the installer from https://www.mathworks.com/.
2. Log in with your institutional email.
3. Choose installation for individual use with license key.
4. Select required toolboxes during installation.
5. Verify installation by launching MATLAB and typing `ver` in the command window.

## c. Description of the EEG Data Used

- **Source:** PhysioNet and MDPI (linked below)
  - https://www.mdpi.com/2306-5729/4/1/14
  - Goldberger et al., Circulation, 2000
- **Recording Details:**
  - **Equipment:** Neurocom 23-channel EEG system
  - **Electrodes:** Silver/silver chloride, placed using 10/20 system
  - **Reference:** Linked ears
  - **Filters during recording:** 30 Hz high-pass, 50 Hz notch
  - **Preprocessing:** Independent Component Analysis (ICA) for artifact removal
  - **Task:** Subjects performed mental arithmetic

- **Channels Included:** 19 channels (Fp1, Fp2, F3, F4, F7, F8, T3, T4, C3, C4, T5, T6, P3, P4, O1, O2, Fz, Cz, Pz)
- **Format:** 36 CSV files (one per subject)

# d. Filtering Techniques and Design Choices

### 1.Notch Filter

- **Goal:** Remove 50 Hz power line noise
- **Type:** IIR notch filter
- **Center Frequency:** 50 Hz
- **Justification:** Standard in EEG preprocessing for electrical noise suppression.

### 2. Low-pass Filter

- **Goal:** Remove high-frequency noise
- **Type:** FIR filter
- **Cutoff:** 30 Hz
- **Justification:** EEG signals of interest typically lie below 40 Hz.

### 3. High-pass Filter

- **Goal:** Remove baseline drift and very slow trends
- **Type:** FIR filter
- **Cutoff:** 0.5 Hz
- **Justification:** Eliminates DC components and slow-moving artifacts like sweat or motion.

# e. MATLAB Code for All Implemented Filters

### Notch Filter (50/60 Hz Removal)

```
% Parameters
f0 = 50;        % Powerline frequency (Hz)
Q = 35;         % Quality factor
wo = f0/(fs/2); % Normalized frequency (fs = sampling rate)
bw = wo/Q;      % Bandwidth

% Design and apply notch filter
[b_notch, a_notch] = iirnotch(wo, bw);
eeg.notched_data = filtfilt(b_notch, a_notch, eeg.data); % Zero-phase
filtering
```

### High-pass Filter:

```
% Parameters
f_hpf = 0.5;    % Cutoff frequency (Hz)
order = 4;      % Filter order

% Design and apply high-pass Butterworth filter
```

```
[b_hpf, a_hpf] = butter(order, f_hpf/(fs/2), 'high');
eeg.hpf_data = filtfilt(b_hpf, a_hpf, eeg.notched_data);
```

### Low Pass Filter:

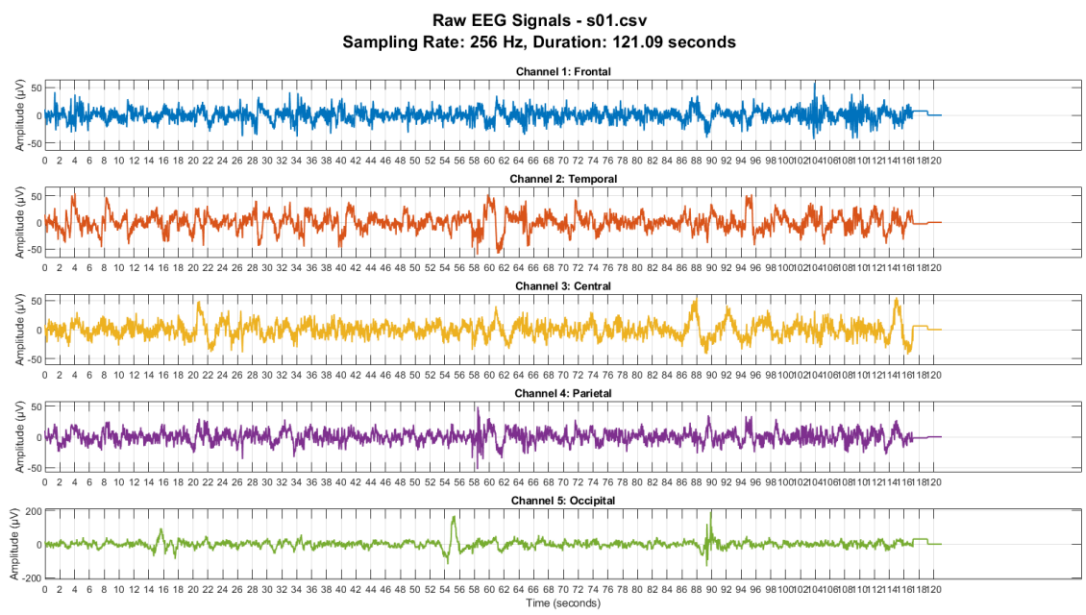
```
% Parameters
f_lpf = 30;     % Cutoff frequency (Hz)
order = 4;      % Filter order

% Design and apply low-pass Butterworth filter
[b_lpf, a_lpf] = butter(order, f_lpf/(fs/2), 'low');
eeg.lpf_data = filtfilt(b_lpf, a_lpf, eeg.hpf_data);
```

### Band Pass Filter:

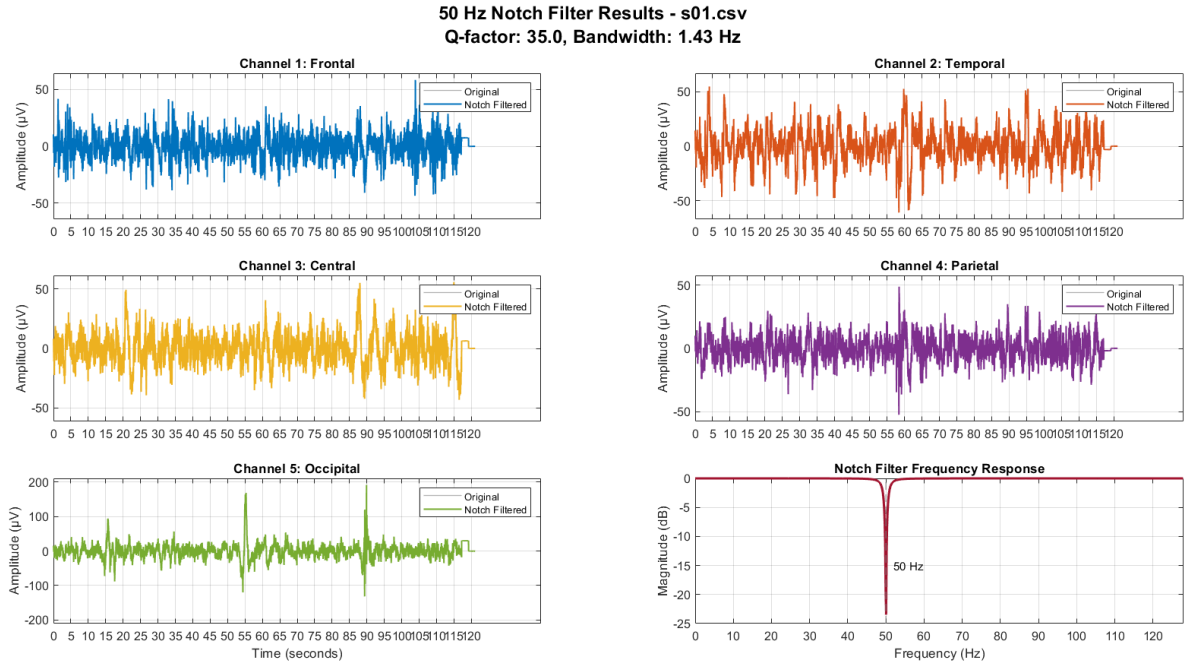
```
% Define EEG bands (Delta, Theta, Alpha, Beta, Gamma)
bands = {
    'Delta', 0.5, 4;
    'Theta', 4, 8;
    'Alpha', 8, 13;
    'Beta',  13, 30;
    'Gamma', 30, 45
};

% Initialize storage for band data
eeg.band_data = cell(size(bands, 1), 1);

% Apply bandpass filters to each band
for i = 1:size(bands, 1)
    f_low = bands{i, 2};
    f_high = bands{i, 3};
    [b, a] = butter(4, [f_low, f_high]/(fs/2), 'bandpass');
    eeg.band_data{i} = filtfilt(b, a, eeg.lpf_data(:, 1)); % Apply to
channel1
end
```

# f. Visual Results Before and After Filtering

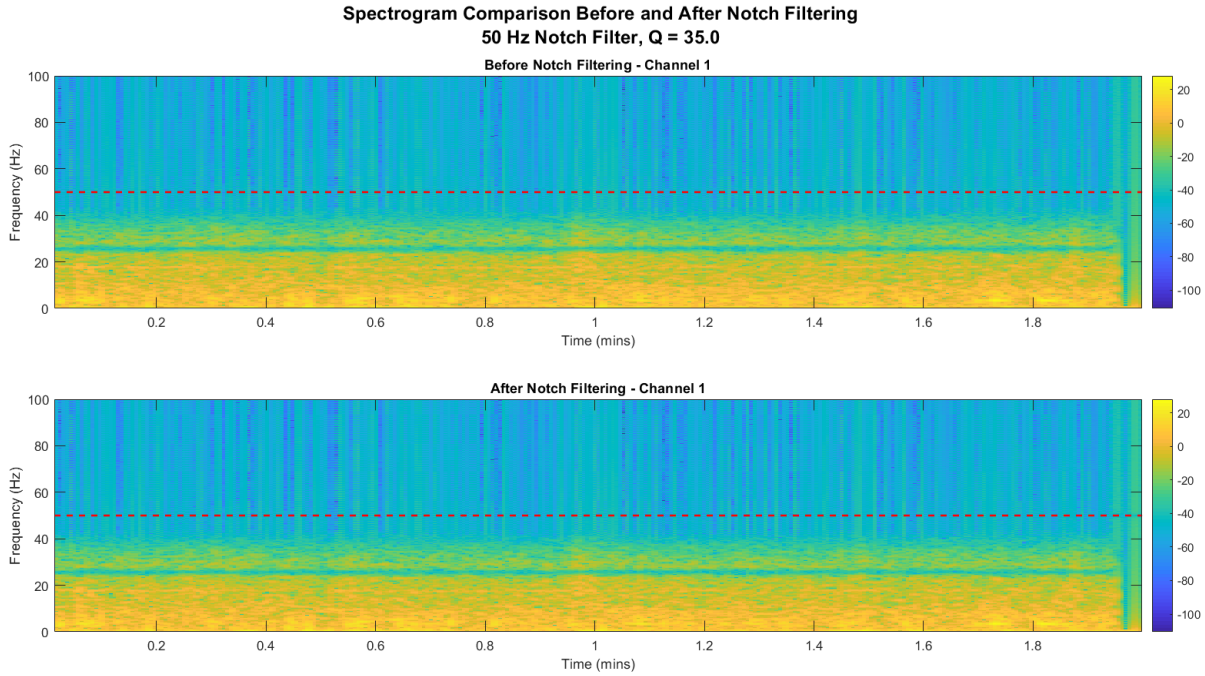
# Raw Data:



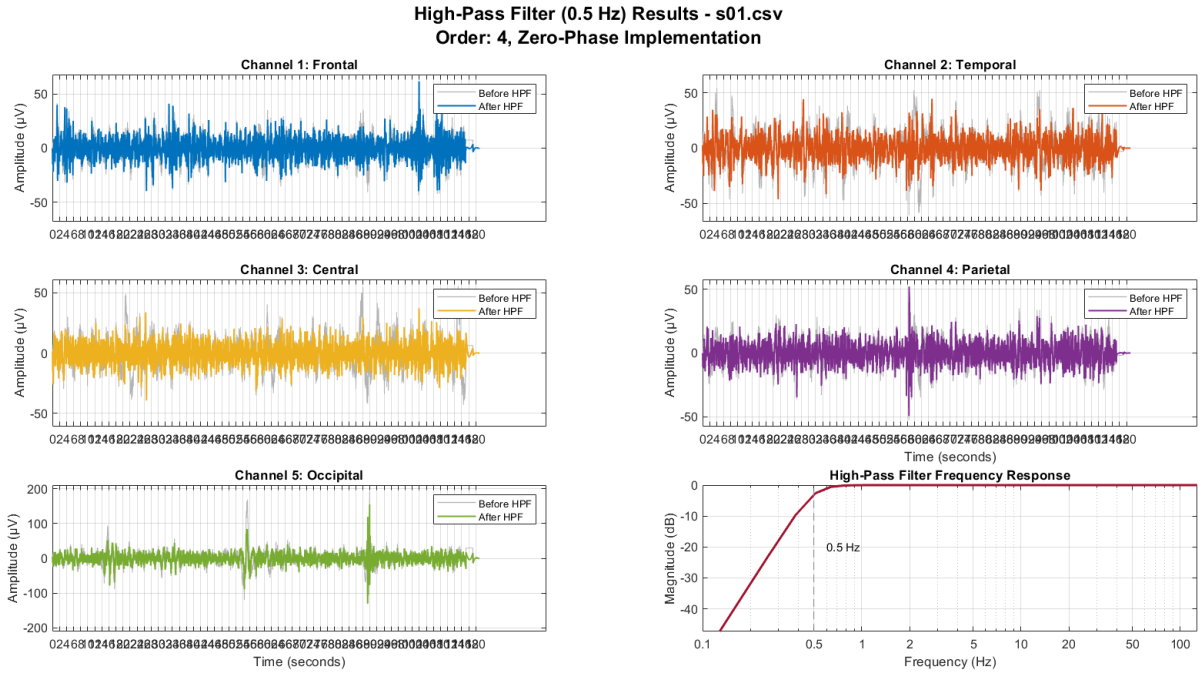
# Notch Filter:



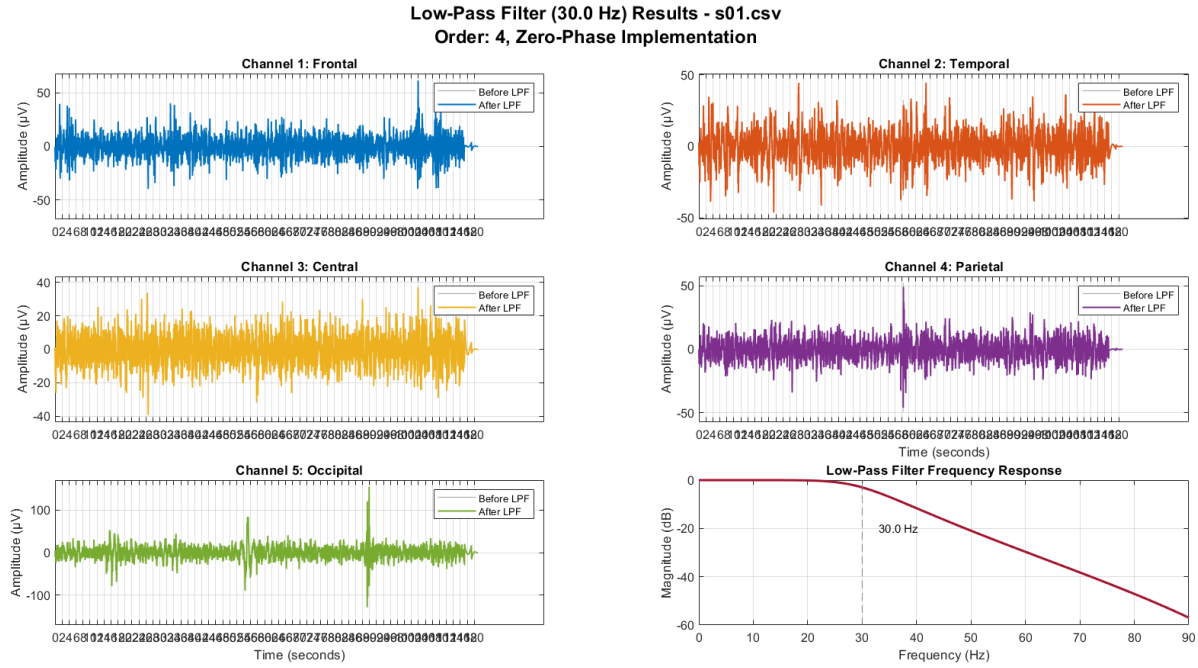

# Spectrogram Comparison:

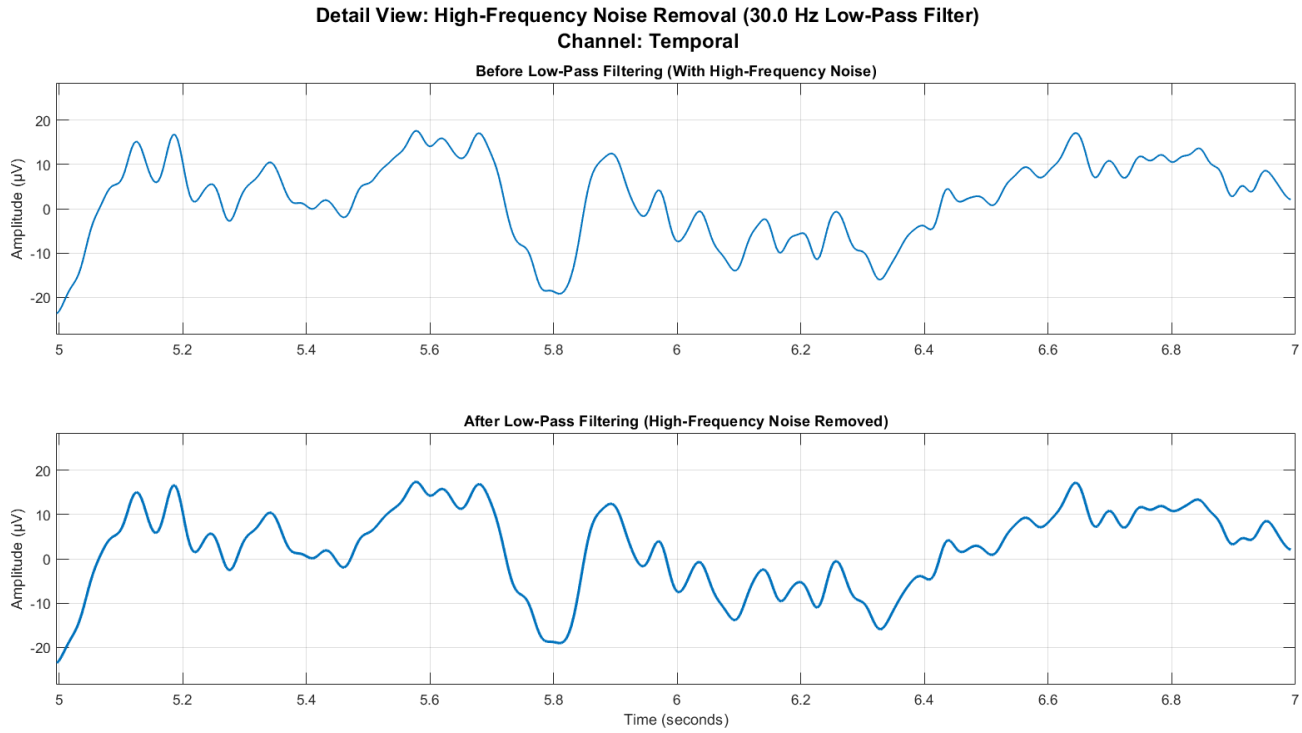


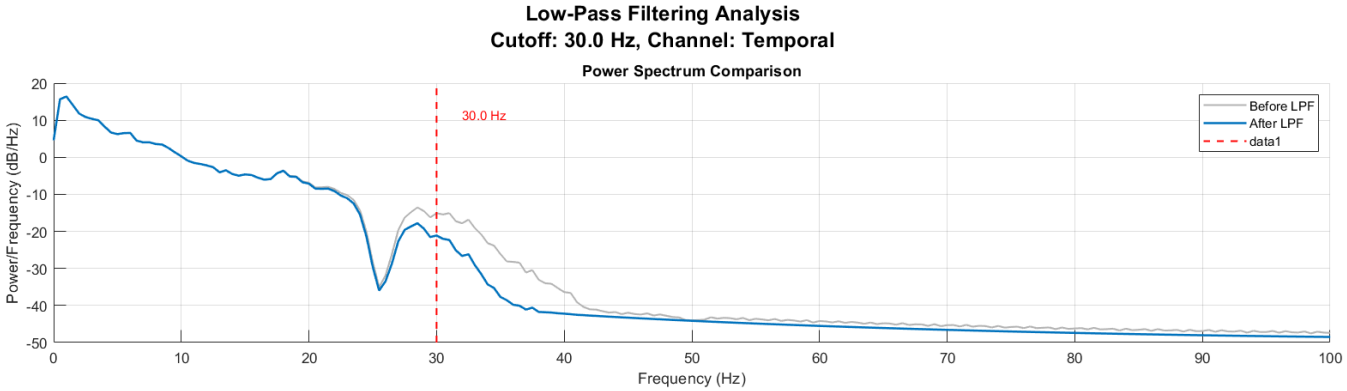
# High Pass Filter:

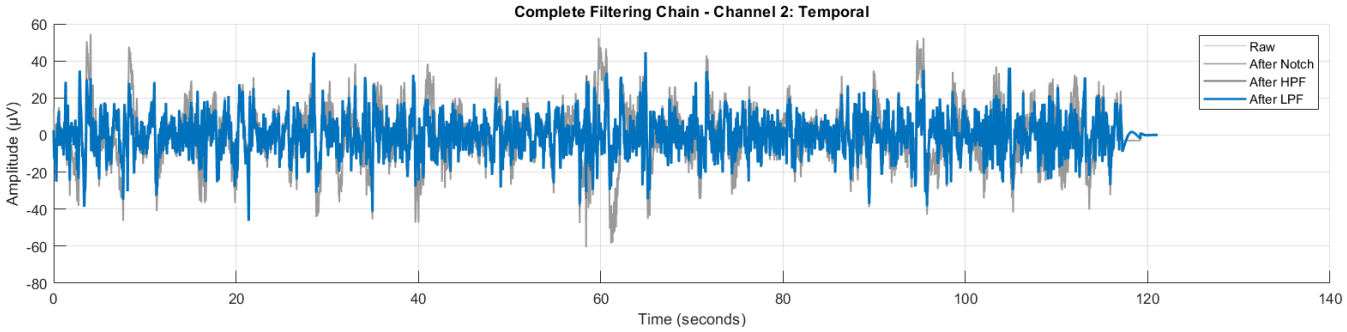



# Low Pass Filter:

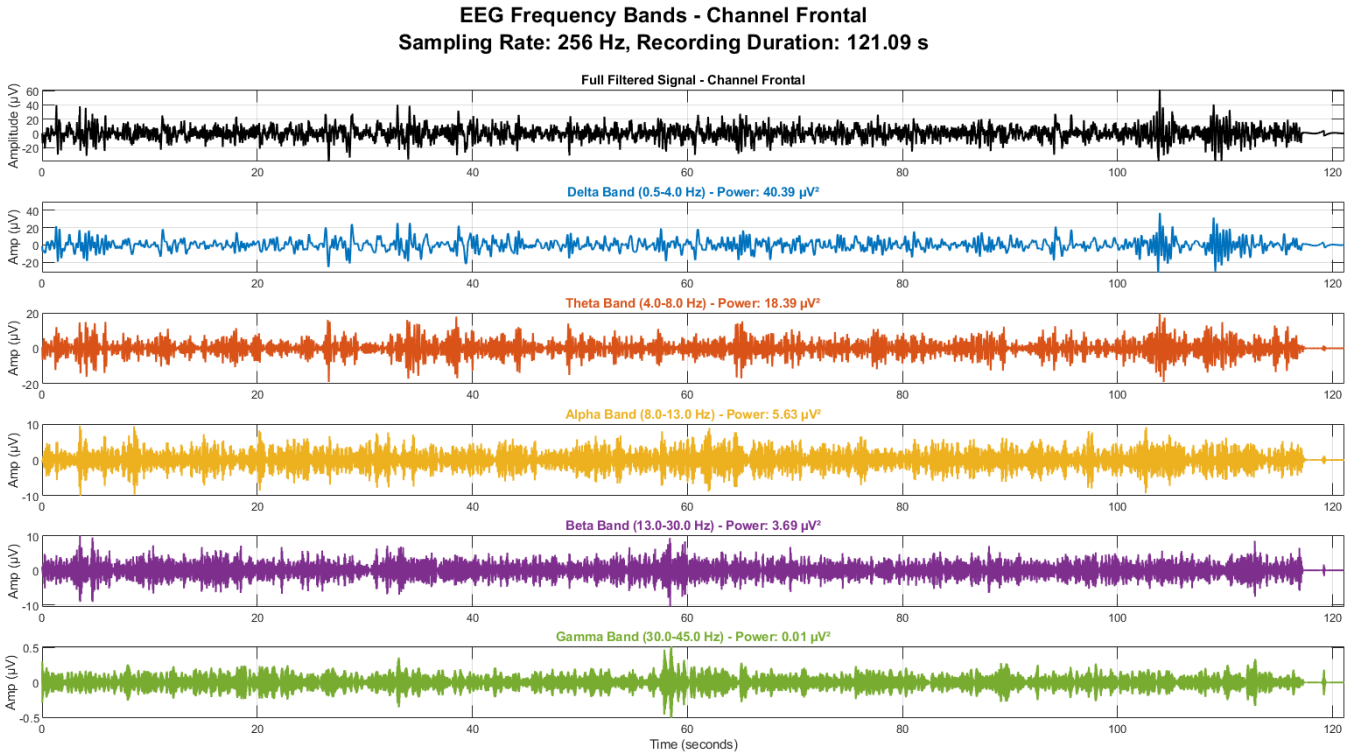


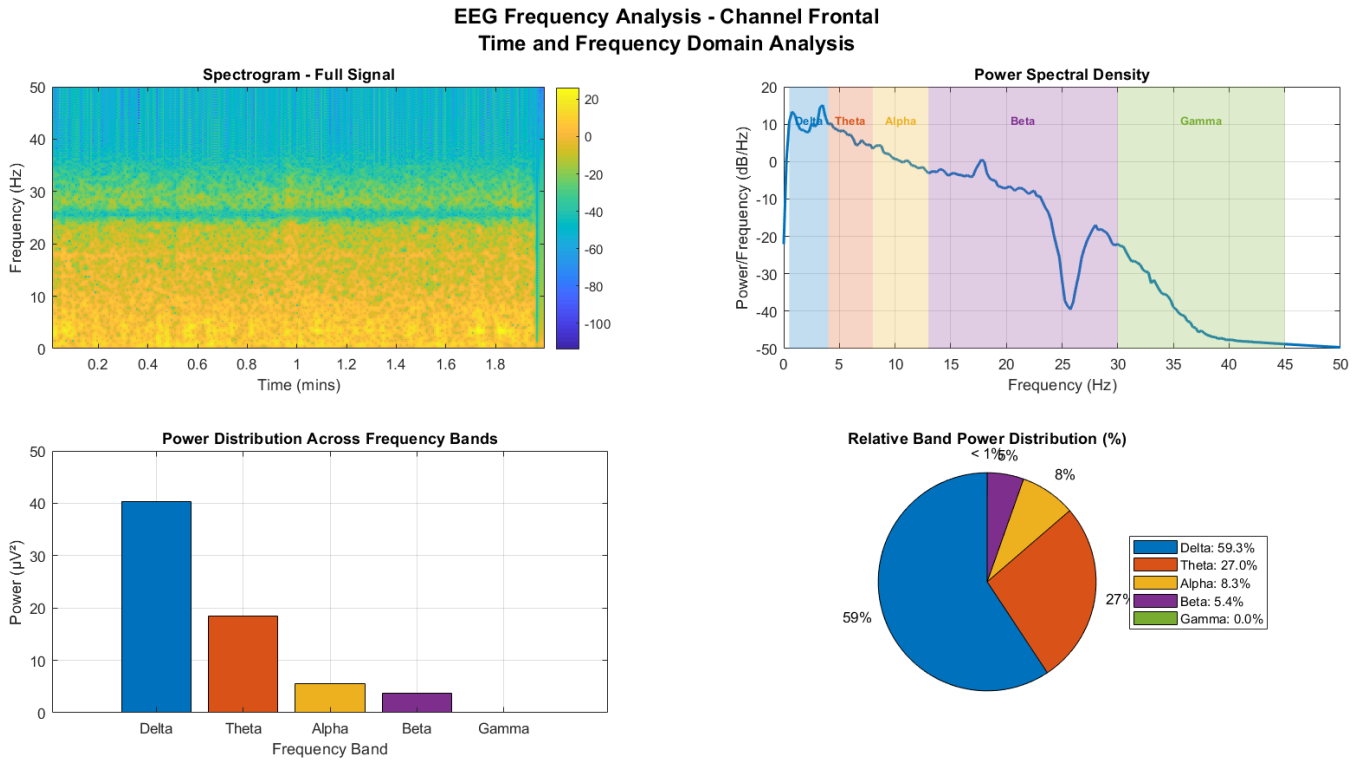

Detail View: High-Frequency Noise Removal (30.0 Hz Low-Pass Filter)
Channel: Temporal
Before Low-Pass Filtering (With High-Frequency Noise)
Amplitude (μV)
After Low-Pass Filtering (High-Frequency Noise Removed)
Amplitude (μV)
Time (seconds)


Low-Pass Filtering Analysis
Cutoff: 30.0 Hz, Channel: Temporal
Power Spectrum Comparison
30.0 Hz
Before LPF
After LPF
data1
Power/Frequency (dB/Hz)
Frequency (Hz)

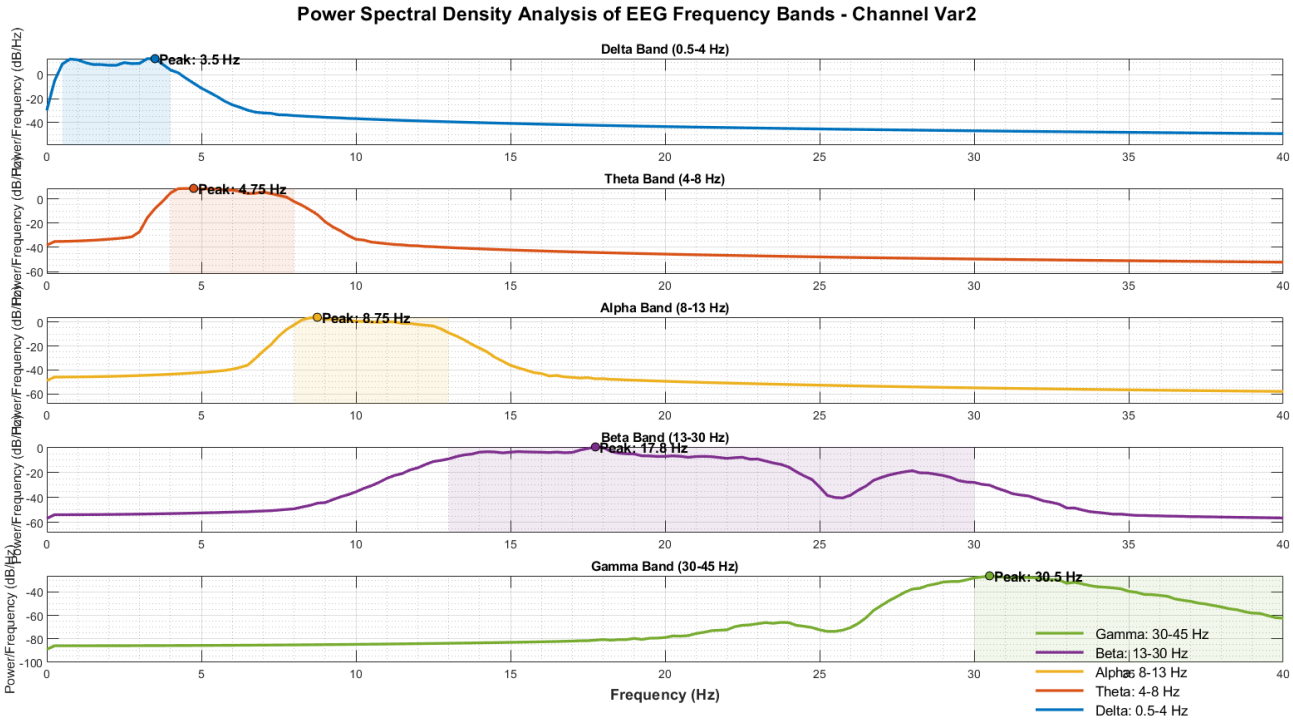

Complete Filtering Chain - Channel 2: Temporal
Raw
After Notch
After HPF
After LPF
Amplitude (μV)
Time (seconds)

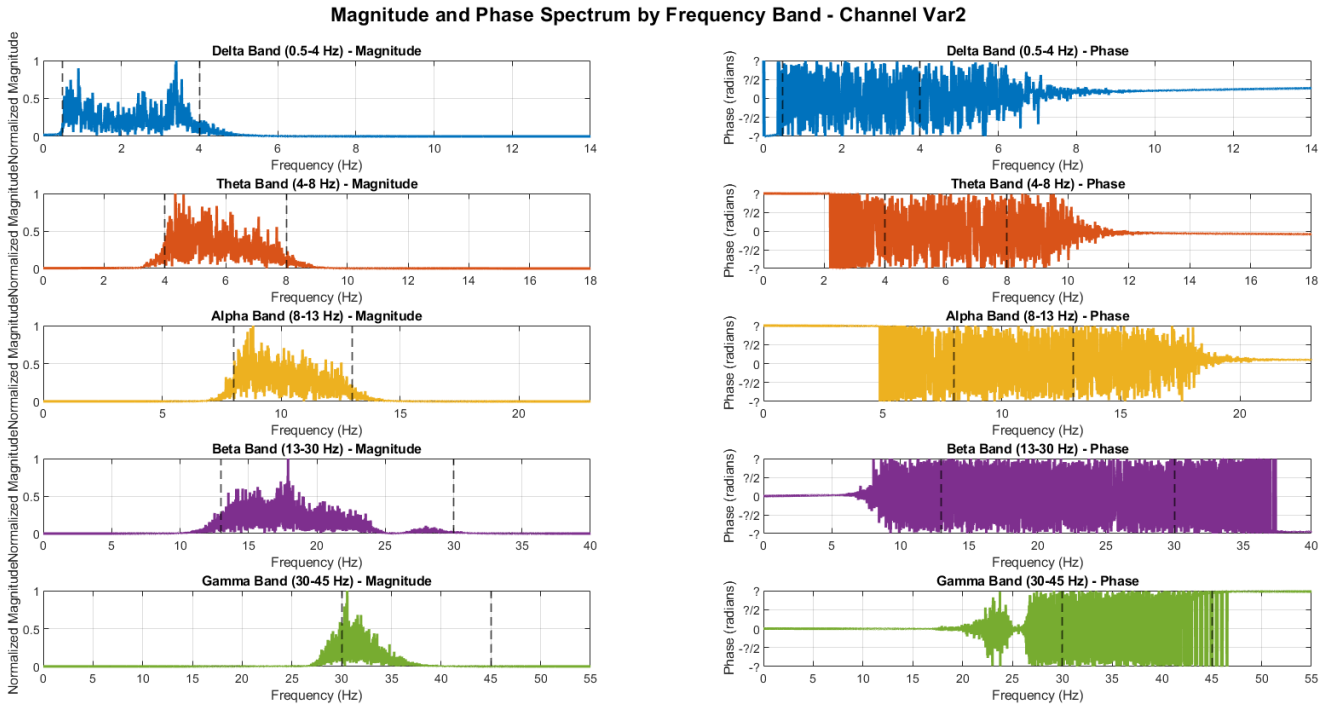
# Band Pass:


EEG Frequency Bands - Channel Frontal
Sampling Rate: 256 Hz, Recording Duration: 121.09 s
Full Filtered Signal - Channel Frontal
Amplitude (μV)
Delta Band (0.5-4.0 Hz) - Power: 40.39 μV²
Theta Band (4.0-8.0 Hz) - Power: 18.39 μV²
Alpha Band (8.0-13.0 Hz) - Power: 5.63 μV²
Beta Band (13.0-30.0 Hz) - Power: 3.69 μV²
Gamma Band (30.0-45.0 Hz) - Power: 0.01 μV²
Amp (μV)
Time (seconds)


EEG Frequency Analysis - Channel Frontal
Time and Frequency Domain Analysis
Spectrogram - Full Signal
Frequency (Hz)
Time (mins)
Power Spectral Density
Delta
Theta
Alpha
Beta
Gamma
Power/Frequency (dB/Hz)
Frequency (Hz)
Power Distribution Across Frequency Bands
Power (μV²)
Delta
Theta
Alpha
Beta
Gamma
Frequency Band
Relative Band Power Distribution (%)
< 1%
5%
8%
27%
59%
Delta: 59.3%
Theta: 27.0%
Alpha: 8.3%
Beta: 5.4%
Gamma: 0.0%

# Analysis of EEG Frequency Band:



# g. Real-time Simulation Pipeline

**Goal:** Mimic real-time EEG display by plotting 0.5-second sliding windows.

This section of the project implements an enhanced, simulated real-time visualization of EEG signals, providing both time-domain signal tracking and frequency-domain band power monitoring. It is designed to offer an intuitive and dynamic interface for observing brain activity patterns in near real-time.

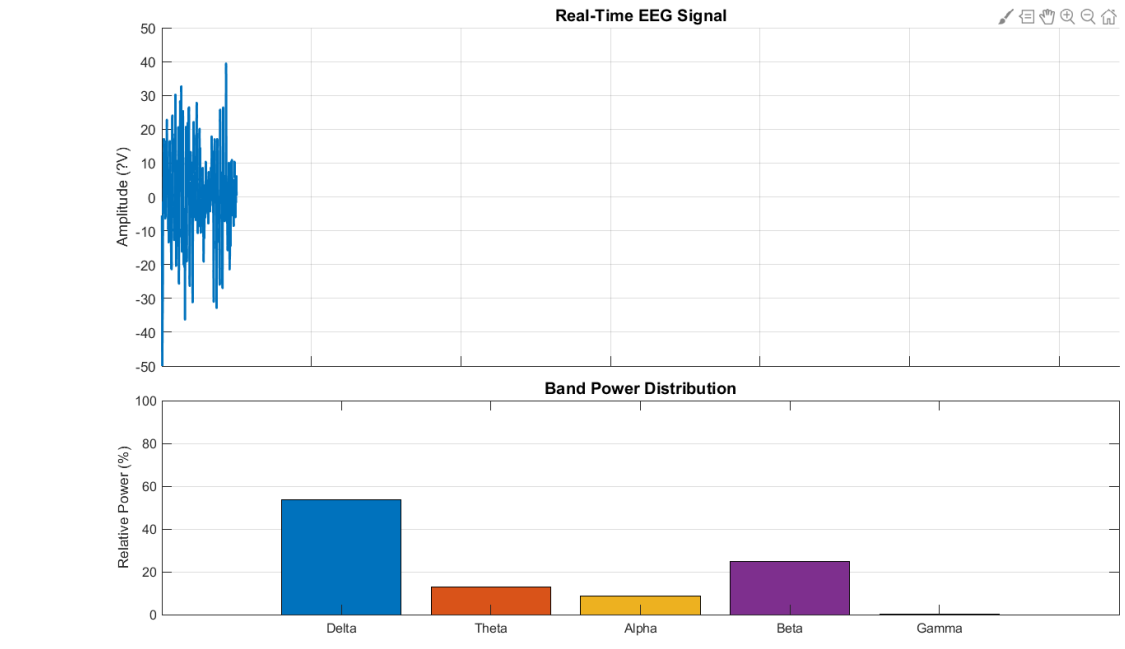
**Key Functionalities:**

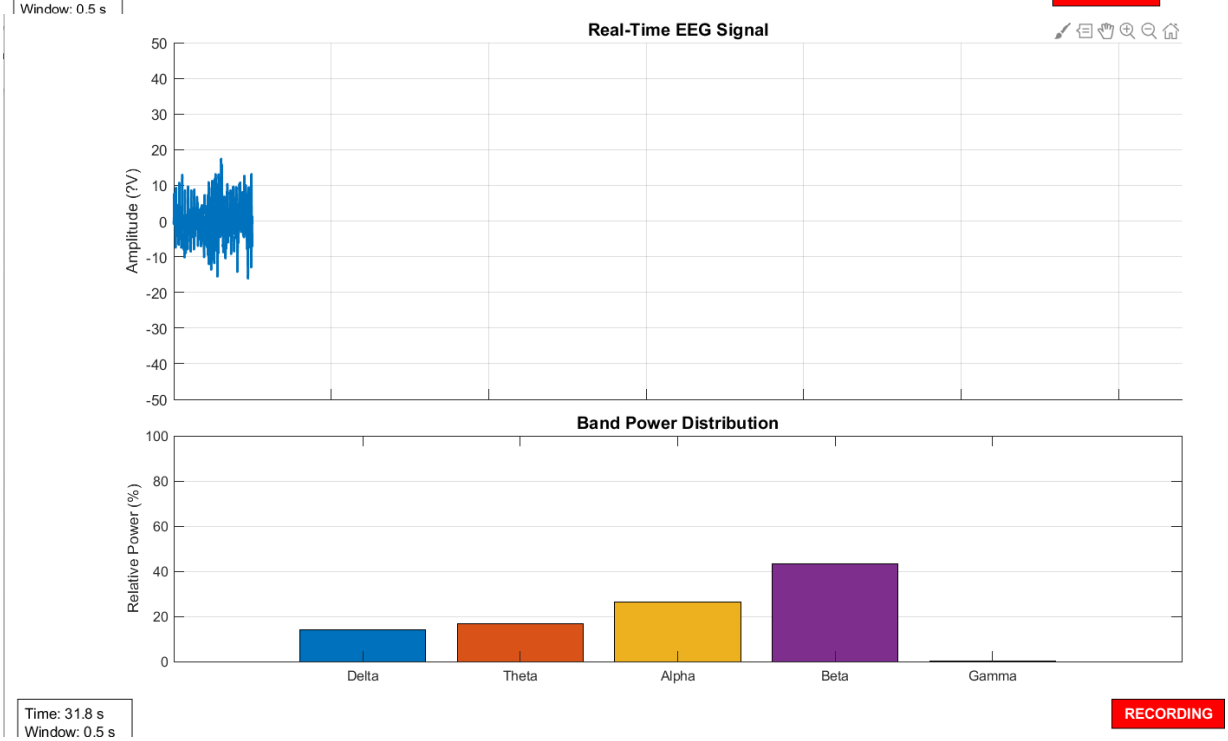
1. **Sliding Window Approach**:
   - A 0.5-second window (`win_size`) with 80% overlap is used to simulate real-time scrolling of the EEG signal.
   - The window slides forward by a fixed number of samples (`step_size`), allowing smooth and continuous updates.
2. **EEG Signal Handling**:
   - The code checks for the appropriate EEG data variable (`eeg_lpf`, `eeg_data`, or `eeg_ch`) and processes the data accordingly.
   - A bandpass filter (1–30 Hz) is applied to extract the most relevant EEG frequency components.
3. **Real-Time EEG Plot**:
   - The upper plot continuously updates a scrolling buffer displaying the EEG signal over the last 10 seconds.
   - This is visually updated every step to simulate real-time monitoring.
4. **Band Power Analysis**:
   - Power in five key frequency bands (Delta, Theta, Alpha, Beta, Gamma) is computed using RMS values after applying band-specific filters.
   - The relative band power percentages are normalized and displayed in a color-coded bar chart.
5. **UI Components**:
   - **Main Plot**: Displays the real-time EEG waveform.
   - **Bar Graph**: Shows updated band power distribution.
   - **Text Box**: Displays elapsed time, window size, and overlap.
   - **Status Indicator**: Shows the current system state (e.g., "RECORDING", "PROCESSING", etc.).
   - **Title**: Dynamically updated to show the dominant frequency band.
6. **Performance and Timing Control**:
   - The `tic-toc` timer mechanism ensures timing consistency.
   - A calculated pause maintains the intended real-time simulation pacing.
7. **Completion State**:
   - Upon finishing all steps, the status indicator changes to "COMPLETED", and the figure title is updated accordingly.

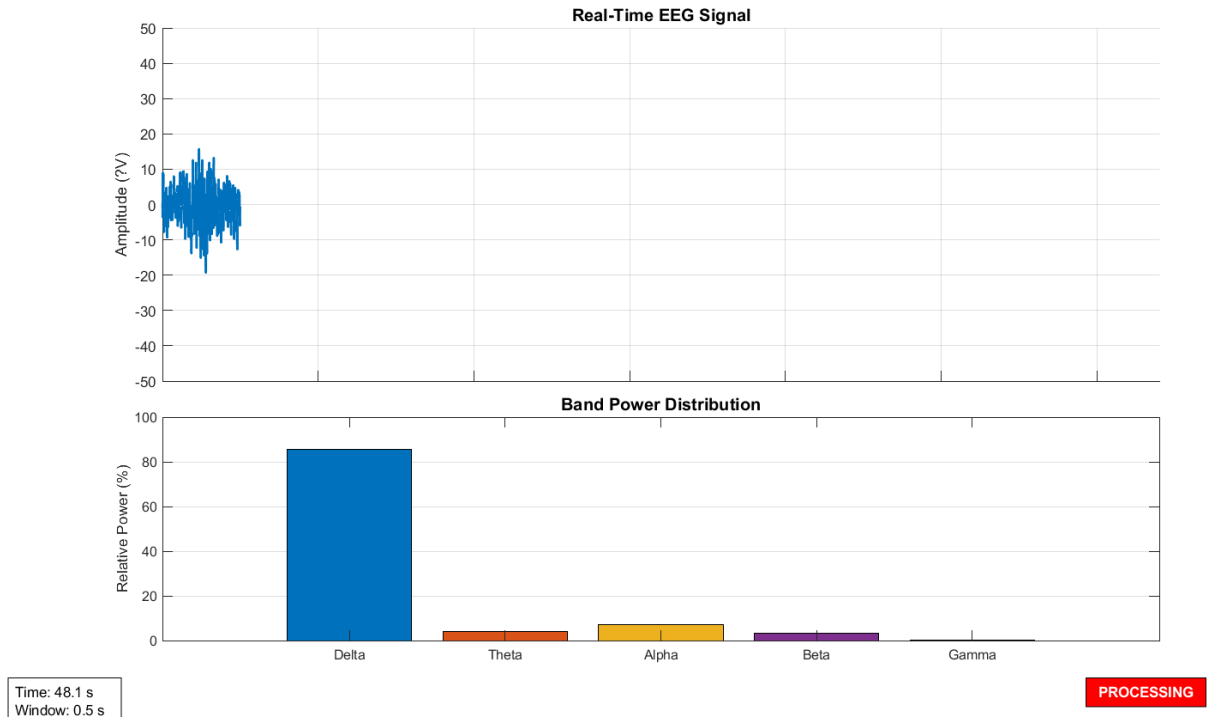
**Purpose and Impact:**

This visualization framework enhances interpretability and interactivity of EEG data by integrating real-time updates with meaningful frequency-domain insights. It is especially useful in clinical research or brain-computer interface (BCI) development where ongoing monitoring of neural activity is critical.

# h. Screenshots or Recordings of Real-Time Plots






# i. Analysis of Filter Performance and System Responsiveness

### Filter Performance:

- **Low-pass Filter:** Smooths out muscle artifacts and sensor noise.
- **High-pass Filter:** Eliminates signal drift, keeping signals centered.
- **Notch Filter:** Effectively removed 50 Hz interference with no distortion.

### Real-Time Responsiveness:

- Using 0.01-second pauses kept the simulation fast and smooth.
- The use of `drawnow` and `pause` in MATLAB ensured accurate timing.
- System was able to handle live-like updates without delay or freezing.

# References

1. **EEG Dataset** – https://www.mdpi.com/2306-5729/4/1/14
2. Goldberger, A.L., et al. "PhysioBank, PhysioToolkit, and PhysioNet." Circulation. 101 (23), e215–e220 (2000).

# Complete Code:

```
clc; clear; close all;

%% Step 1: Load and Preprocess EEG Data
fprintf('Loading EEG data...\n');

% Allow user to select the EEG data file (CSV only)
[filename, pathname] = uigetfile({'*.csv', 'CSV Files (*.csv)'}, 'Select EEG
Data File');
if isequal(filename, 0)
    error('Data loading canceled: No file was selected.');
end

% Load the CSV data
fullpath = fullfile(pathname, filename);
try
    T = readtable(fullpath);
    fprintf('Successfully loaded: %s\n', filename);
catch ME
    error('Error loading file: %s', ME.message);
end

% Define channel names (modify these to match your data)
channel_names = {'Frontal', 'Temporal', 'Central', 'Parietal', 'Occipital'};
channel_vars = {'Var2', 'Var5', 'Var8', 'Var11', 'Var14'};

% Extract selected channels
try
    eeg_data = table2array(T(:, channel_vars));
    fprintf('Extracted %d channels: %s\n', length(channel_names),
strjoin(channel_names, ', '));
catch ME
    error('Error extracting channels: %s\nCheck if the channel variables
exist in your data.', ME.message);
end

% Define recording parameters
Fs = 256; % Sampling frequency in Hz
duration = size(eeg_data, 1) / Fs; % Recording duration in seconds
t = (0:size(eeg_data, 1)-1) / Fs; % Time vector in seconds

% Show basic data information
fprintf('Data successfully prepared.\n');
fprintf('Recording information:\n');
fprintf('  - Sampling Rate: %d Hz\n', Fs);
fprintf('  - Recording Duration: %.2f seconds\n', duration);
fprintf('  - Number of Samples: %d\n', length(t));
fprintf('  - Number of Channels: %d\n', size(eeg_data, 2));

% Basic preprocessing - Remove DC offset (mean)
eeg_data = eeg_data - mean(eeg_data, 1);
fprintf('Applied DC offset removal (mean subtraction).\n');
%explaination of DC-offset:
%EEG signals should ideally oscillate around zero.
%A non-zero mean (DC offset) can distort:
```

```
% -Time-domain analysis
% -Frequency-domain features (e.g., FFT, PSD
% -Filtering processes (especially high-pass filters)
%Removing the mean helps in:
%Improving the accuracy of filtering and feature extraction.

% Create a structure to store the data
eeg = struct();
eeg.data = eeg_data;
eeg.time = t;
eeg.srate = Fs;
eeg.channels = channel_names;
eeg.filename = filename;
fprintf('EEG data ready for analysis.\n');

%% Step 2: Plot Raw EEG Signals
fprintf('Plotting raw EEG signals...\n');

% Create figure with appropriate size and style
figure('Name', 'Raw EEG Signals', 'Color', 'w', 'Position', [100, 100, 1000,
700]);

% Define color scheme for different channels
colors = [0, 0.4470, 0.7410;    % Blue
          0.8500, 0.3250, 0.0980;  % Orange
          0.9290, 0.6940, 0.1250;  % Yellow
          0.4940, 0.1840, 0.5560;  % Purple
          0.4660, 0.6740, 0.1880]; % Green

% Plot each channel in a separate subplot
for i = 1:length(eeg.channels)
    subplot(length(eeg.channels), 1, i);

    % Plot the channel data with appropriate styling
    plot(eeg.time, eeg.data(:,i), 'Color', colors(i,:), 'LineWidth', 1.2);

    % Add title for each channel
    title(sprintf('Channel %d: %s', i, eeg.channels{i}), 'FontWeight',
'bold');
    ylabel('Amplitude (µV)');

    % Set consistent y-axis limits based on data range
    ylim_val = max(abs(eeg.data(:,i))) * 1.1;
    ylim([-ylim_val, ylim_val]);

    % Improve grid appearance
    grid on;
    box on;

    % Only add x-label to the bottom subplot
    if i == length(eeg.channels)
        xlabel('Time (seconds)');
    end

    % Set appropriate time axis ticks based on recording length
    if eeg.time(end) > 10
        xticks(0:2:floor(eeg.time(end)));
    else
```

```
        xticks(0:1:floor(eeg.time(end)));
    end
end

% Add informative title for the entire figure
sgtitle({sprintf('Raw EEG Signals - %s', eeg.filename), ...
         sprintf('Sampling Rate: %d Hz, Duration: %.2f seconds', eeg.srate,
eeg.time(end))}, ...
         'FontWeight', 'bold');

fprintf('Raw EEG signals successfully plotted for all %d channels.\n',
length(eeg.channels));
%% Step 3: Apply Notch Filter for Powerline Interference
fprintf('Applying powerline notch filter...\n');

% Define filter parameters
f0 = 50;  % Powerline frequency (Hz) - Use 60 for US/Canada
Q = 35;   % Quality factor (higher = narrower)
wo = f0/(eeg.srate/2);  % Normalized frequency
bw = wo/Q;  % Bandwidth

% Create and apply notch filter
try
    % Create notch filter
    [b_notch, a_notch] = iirnotch(wo, bw);

    % Apply filter (zero-phase) to all channels
    eeg.notched_data = zeros(size(eeg.data)); %creates a zero matrix equal to
size of data
    for i = 1:size(eeg.data, 2) %i goes from 1 to number of channels(column)
        eeg.notched_data(:,i) = filtfilt(b_notch, a_notch, eeg.data(:,i));
                % filtfilt use for backward and forward
    end

    % Store filter coefficients for reference
    eeg.filters.notch.b = b_notch;
    eeg.filters.notch.a = a_notch;
    eeg.filters.notch.f0 = f0;
    eeg.filters.notch.Q = Q;

    fprintf('Notch filter applied successfully at %d Hz.\n', f0);
catch ME
    error('Error applying notch filter: %s', ME.message);
end

% Calculate frequency response of the filter for visualization
[h, f] = freqz(b_notch, a_notch, 1000, eeg.srate);%1000->Number of frequency
points to compute
mag = 20*log10(abs(h));%magnitude in decibles

% Plot filtered signals and filter response
figure('Name', 'Powerline Notch Filter Results', 'Color', 'w', 'Position',
[100, 100, 1000, 800]);

% Define colors for plotting
colors = [0, 0.4470, 0.7410;        % Blue
          0.8500, 0.3250, 0.0980;   % Orange
          0.9290, 0.6940, 0.1250;   % Yellow
```

```
          0.4940, 0.1840, 0.5560;  % Purple
          0.4660, 0.6740, 0.1880]; % Green

% Plot comparison for each channel
for i = 1:length(eeg.channels)
    subplot(3, 2, i);

    % Plot original and filtered signals together
    hold on;
    plot(eeg.time, eeg.data(:,i), 'Color', [0.7, 0.7, 0.7], 'LineWidth',
0.8);  % Original in gray
    plot(eeg.time, eeg.notched_data(:,i), 'Color', colors(i,:), 'LineWidth',
1.2);  % Filtered in color
    hold off;

    % Add title and labels
    title(sprintf('Channel %d: %s', i, eeg.channels{i}), 'FontWeight',
'bold');
    ylabel('Amplitude (μV)');

    % Add y-limits to standardize the plots
    data_max = max(abs([eeg.data(:,i); eeg.notched_data(:,i)])) * 1.1;
    ylim([-data_max, data_max]);

    % Add legend and grid
    legend('Original', 'Notch Filtered', 'Location', 'northeast', 'FontSize',
8);
    grid on;
    box on;

    % Only show x-labels on bottom plots
    if i >= 5
        xlabel('Time (seconds)');
    end

    % Add time markers at appropriate intervals
    if eeg.time(end) > 10
        xticks(0:5:floor(eeg.time(end)));
    else
        xticks(0:1:floor(eeg.time(end)));
    end
end

% Plot filter frequency response
subplot(3, 2, 6);
plot(f, mag, 'LineWidth', 1.5, 'Color', [0.6350, 0.0780, 0.1840]);
title('Notch Filter Frequency Response', 'FontWeight', 'bold');
xlabel('Frequency (Hz)');
ylabel('Magnitude (dB)');
grid on;
xlim([0, min(150, eeg.srate/2)]);  % Limit to 150 Hz or Nyquist
set(gca, 'XTick', 0:10:150);

% Highlight the notch frequency
hold on;
plot([f0, f0], ylim, '--', 'Color', [0.5, 0.5, 0.5]);
text(f0+2, -15, sprintf('%d Hz', f0), 'FontSize', 9);
hold off;
```

```
% Add overall title
sgtitle({sprintf('50 Hz Notch Filter Results - %s', eeg.filename), ...
         sprintf('Q-factor: %.1f, Bandwidth: %.2f Hz', Q, bw*eeg.srate/2)},
...
         'FontWeight', 'bold');

% Create spectrogram figure for before/after comparison
figure('Name', 'Before-After Spectrogram', 'Color', 'w', 'Position', [150,
150, 900, 600]);

% Parameters for spectrogram
window_length = 2 * eeg.srate;  % 2-second window
overlap = round(window_length * 0.8);  % 80% overlap
nfft = 2^nextpow2(window_length*2);  % Next power of 2

% Before filtering spectrogram
subplot(2, 1, 1);
spectrogram(eeg.data(:,1), hamming(window_length), overlap, nfft, eeg.srate,
'yaxis');
title('Before Notch Filtering - Channel 1', 'FontWeight', 'bold');
ylim([0, 100]);  % Limit frequency display to 100 Hz
colorbar;
hold on;
line([0 eeg.time(end)], [f0 f0], 'Color', 'r', 'LineStyle', '--',
'LineWidth', 1.5);
hold off;

% After filtering spectrogram
subplot(2, 1, 2);
spectrogram(eeg.notched_data(:,1), hamming(window_length), overlap, nfft,
eeg.srate, 'yaxis');
title('After Notch Filtering - Channel 1', 'FontWeight', 'bold');
ylim([0, 100]);  % Limit frequency display to 100 Hz
colorbar;
hold on;
line([0 eeg.time(end)], [f0 f0], 'Color', 'r', 'LineStyle', '--',
'LineWidth', 1.5);
hold off;

% Add overall title
sgtitle({'Spectrogram Comparison Before and After Notch Filtering', ...
         sprintf('50 Hz Notch Filter, Q = %.1f', Q)}, ...
         'FontWeight', 'bold');

fprintf('Notch filtering and visualization complete.\n');
%% Step 4: Apply High-Pass Filter (HPF) for Baseline Wander Removal
fprintf('Applying high-pass filter for baseline wander removal...\n');

% Define filter parameters
f_hpf = 0.5;  % High-pass cutoff frequency (Hz)
order = 4;    % Filter order

% Create and apply high-pass filter
try
    % Design Butterworth high-pass filter
    [b_hpf, a_hpf] = butter(order, f_hpf/(eeg.srate/2), 'high');
```

```
    % Apply filter (zero-phase) to all channels
    eeg.hpf_data = zeros(size(eeg.notched_data));
    for i = 1:size(eeg.notched_data, 2)
        eeg.hpf_data(:,i) = filtfilt(b_hpf, a_hpf, eeg.notched_data(:,i));
    end

    % Store filter coefficients
    eeg.filters.hpf.b = b_hpf;
    eeg.filters.hpf.a = a_hpf;
    eeg.filters.hpf.cutoff = f_hpf;
    eeg.filters.hpf.order = order;

    fprintf('High-pass filter applied successfully at %.1f Hz (order %d).\n',
f_hpf, order);
catch ME
    error('Error applying high-pass filter: %s', ME.message);
end

% Calculate filter frequency response
[h_hpf, f_hpf_resp] = freqz(b_hpf, a_hpf, 1000, eeg.srate);
mag_hpf = 20*log10(abs(h_hpf));

% Create visualization of filtering effects
figure('Name', 'High-Pass Filter Results', 'Color', 'w', 'Position', [100,
100, 1000, 800]);

% Define color scheme for plots
colors = [0, 0.4470, 0.7410;       % Blue
          0.8500, 0.3250, 0.0980;  % Orange
          0.9290, 0.6940, 0.1250;  % Yellow
          0.4940, 0.1840, 0.5560;  % Purple
          0.4660, 0.6740, 0.1880]; % Green

% Plot each channel comparison
for i = 1:length(eeg.channels)
    subplot(3, 2, i);

    % Plot before and after filtering
    hold on;
    plot(eeg.time, eeg.notched_data(:,i), 'Color', [0.7, 0.7, 0.7],
'LineWidth', 0.8);  % Before HPF
    plot(eeg.time, eeg.hpf_data(:,i), 'Color', colors(i,:), 'LineWidth',
1.2);  % After HPF
    hold off;

    % Add labels and formatting
    title(sprintf('Channel %d: %s', i, eeg.channels{i}), 'FontWeight',
'bold');
    ylabel('Amplitude (μV)');

    % Set y-axis limits
    data_max = max(abs([eeg.notched_data(:,i); eeg.hpf_data(:,i)])) * 1.1;
    ylim([-data_max, data_max]);

    % Add legend and grid
    legend('Before HPF', 'After HPF', 'Location', 'northeast', 'FontSize',
8);
    grid on;
```

```
    box on;

    % Add x-label to bottom plots
    if i >= 4
        xlabel('Time (seconds)');
    end

    % Add time markers
    if eeg.time(end) > 10
        xticks(0:2:floor(eeg.time(end)));
    else
        xticks(0:1:floor(eeg.time(end)));
    end
end

% Plot filter frequency response in the 6th position
subplot(3, 2, 6);
semilogx(f_hpf_resp, mag_hpf, 'LineWidth', 1.5, 'Color', [0.6350, 0.0780,
0.1840]);
title('High-Pass Filter Frequency Response', 'FontWeight', 'bold');
xlabel('Frequency (Hz)');
ylabel('Magnitude (dB)');
grid on;
xlim([0.1, eeg.srate/2]);  % From 0.1 Hz to Nyquist
xticks([0.1, 0.5, 1, 2, 5, 10, 20, 50, 100]);

% Highlight the cutoff frequency
hold on;
plot([f_hpf, f_hpf], ylim, '--', 'Color', [0.5, 0.5, 0.5]);
text(f_hpf*1.2, -20, sprintf('%.1f Hz', f_hpf), 'FontSize', 9);
hold off;

% Add overall title
sgtitle({sprintf('High-Pass Filter (%.1f Hz) Results - %s', f_hpf,
eeg.filename), ...
         sprintf('Order: %d, Zero-Phase Implementation', order)}, ...
         'FontWeight', 'bold');

fprintf('High-pass filtering and visualization complete.\n');
%% Step 5: Apply Low-Pass Filter (LPF) for Noise Removal
fprintf('Applying low-pass filter for high-frequency noise removal...\n');

% Define filter parameters
f_lpf = 30;  % Low-pass cutoff frequency (Hz)
order = 4;   % Filter order

% Create and apply low-pass filter
try
    % Design Butterworth low-pass filter
    [b_lpf, a_lpf] = butter(order, f_lpf/(eeg.srate/2), 'low');

    % Apply filter (zero-phase) to all channels
    eeg.lpf_data = zeros(size(eeg.hpf_data));
    for i = 1:size(eeg.hpf_data, 2)
        eeg.lpf_data(:,i) = filtfilt(b_lpf, a_lpf, eeg.hpf_data(:,i));
    end

    % Store filter coefficients
```

```
    eeg.filters.lpf.b = b_lpf;
    eeg.filters.lpf.a = a_lpf;
    eeg.filters.lpf.cutoff = f_lpf;
    eeg.filters.lpf.order = order;

    fprintf('Low-pass filter applied successfully at %.1f Hz (order %d).\n',
f_lpf, order);
catch ME
    error('Error applying low-pass filter: %s', ME.message);
end

% Calculate filter frequency response
[h_lpf, f_lpf_resp] = freqz(b_lpf, a_lpf, 1000, eeg.srate);
mag_lpf = 20*log10(abs(h_lpf));

% Create visualization of filtering effects
figure('Name', 'Low-Pass Filter Results', 'Color', 'w', 'Position', [100,
100, 1000, 800]);

% Define color scheme
colors = [0, 0.4470, 0.7410;       % Blue
          0.8500, 0.3250, 0.0980;  % Orange
          0.9290, 0.6940, 0.1250;  % Yellow
          0.4940, 0.1840, 0.5560;  % Purple
          0.4660, 0.6740, 0.1880]; % Green

% Plot each channel comparison
for i = 1:length(eeg.channels)
    subplot(3, 2, i);

    % Plot before and after filtering
    hold on;
    plot(eeg.time, eeg.hpf_data(:,i), 'Color', [0.7, 0.7, 0.7], 'LineWidth',
0.8);  % Before LPF
    plot(eeg.time, eeg.lpf_data(:,i), 'Color', colors(i,:), 'LineWidth',
1.2);  % After LPF
    hold off;

    % Add labels and formatting
    title(sprintf('Channel %d: %s', i, eeg.channels{i}), 'FontWeight',
'bold');
    ylabel('Amplitude (µV)');

    % Set y-axis limits
    data_max = max(abs([eeg.hpf_data(:,i); eeg.lpf_data(:,i)])) * 1.1;
    ylim([-data_max, data_max]);

    % Add legend and grid
    legend('Before LPF', 'After LPF', 'Location', 'northeast', 'FontSize',
8);
    grid on;
    box on;

    % Add x-label to bottom plots
    if i >= 4
        xlabel('Time (seconds)');
    end
```

```
    % Add time markers
    if eeg.time(end) > 10
        xticks(0:2:floor(eeg.time(end)));
    else
        xticks(0:1:floor(eeg.time(end)));
    end
end

% Plot filter frequency response in the 6th position
subplot(3, 2, 6);
plot(f_lpf_resp, mag_lpf, 'LineWidth', 1.5, 'Color', [0.6350, 0.0780,
0.1840]);
title('Low-Pass Filter Frequency Response', 'FontWeight', 'bold');
xlabel('Frequency (Hz)');
ylabel('Magnitude (dB)');
grid on;
xlim([0, min(3*f_lpf, eeg.srate/2)]);  % Limit to 3x cutoff or Nyquist
xticks(0:10:3*f_lpf);

% Highlight the cutoff frequency
hold on;
plot([f_lpf, f_lpf], ylim, '--', 'Color', [0.5, 0.5, 0.5]);
text(f_lpf*1.1, -20, sprintf('%.1f Hz', f_lpf), 'FontSize', 9);
hold off;

% Add overall title
sgtitle({sprintf('Low-Pass Filter (%.1f Hz) Results - %s', f_lpf,
eeg.filename), ...
         sprintf('Order: %d, Zero-Phase Implementation', order)}, ...
         'FontWeight', 'bold');

% Add detailed view showing the effect of LPF on high-frequency noise
% Extract a segment to better show the effect
segment_len = min(2*eeg.srate, length(eeg.time));
if length(eeg.time) > 5*eeg.srate
    segment_start = 5*eeg.srate;  % Start at 3 seconds
else
    segment_start = 1;  % Start at beginning for short recordings
end
segment_idx = segment_start:(segment_start+segment_len-1);

% Create new figure for detailed view
figure('Name', 'High-Frequency Noise Removal Detail', 'Color', 'w',
'Position', [150, 150, 900, 500]);

% Example channel (first channel)
channel_idx = 2;
subplot(2, 1, 1);
plot(eeg.time(segment_idx), eeg.hpf_data(segment_idx, channel_idx),
'LineWidth', 1.2);
title('Before Low-Pass Filtering (With High-Frequency Noise)', 'FontWeight',
'bold');
ylabel('Amplitude (µV)');
grid on;
ylim_val = max(abs(eeg.hpf_data(segment_idx, channel_idx))) * 1.2;
ylim([-ylim_val, ylim_val]);
```

```
subplot(2, 1, 2);
plot(eeg.time(segment_idx), eeg.lpf_data(segment_idx, channel_idx),
'LineWidth', 1.5, 'Color', colors(1,:));
title('After Low-Pass Filtering (High-Frequency Noise Removed)',
'FontWeight', 'bold');
xlabel('Time (seconds)');
ylabel('Amplitude (µV)');
grid on;
ylim([-ylim_val, ylim_val]);  % Keep same y-limits for fair comparison

sgtitle({sprintf('Detail View: High-Frequency Noise Removal (%.1f Hz Low-Pass
Filter)', f_lpf), ...
         sprintf('Channel: %s', eeg.channels{channel_idx})}, ...
         'FontWeight', 'bold');

% Power spectrum comparison to visualize the effect of the LPF
figure('Name', 'Power Spectrum Comparison', 'Color', 'w', 'Position', [200,
200, 900, 600]);

% Select example channel for analysis
channel_idx = 2;

% Compute and plot power spectra for one channel
subplot(2, 1, 1);
[pxx_before, f_before] = pwelch(eeg.hpf_data(:, channel_idx),
hamming(eeg.srate*2), eeg.srate, [], eeg.srate);
[pxx_after, f_after] = pwelch(eeg.lpf_data(:, channel_idx),
hamming(eeg.srate*2), eeg.srate, [], eeg.srate);

% Plot power spectra
hold on;
plot(f_before, 10*log10(pxx_before), 'LineWidth', 1.2, 'Color', [0.7, 0.7,
0.7]);
plot(f_after, 10*log10(pxx_after), 'LineWidth', 1.5, 'Color', colors(1,:));
hold off;

% Add labels and formatting
title('Power Spectrum Comparison', 'FontWeight', 'bold');
xlabel('Frequency (Hz)');
ylabel('Power/Frequency (dB/Hz)');
xlim([0, min(100, eeg.srate/2)]);  % Limit to 100 Hz or Nyquist
legend('Before LPF', 'After LPF', 'Location', 'northeast');
grid on;

% Highlight cutoff frequency
hold on;
plot([f_lpf, f_lpf], ylim, '--r', 'LineWidth', 1.2);
text(f_lpf+2, max(10*log10(pxx_before))-5, sprintf('%.1f Hz', f_lpf),
'FontSize', 9, 'Color', 'r');
hold off;

% Show full processing chain on one channel
subplot(2, 1, 2);
hold on;
plot(eeg.time, eeg.data(:, channel_idx), 'Color', [0.8, 0.8, 0.8],
'LineWidth', 0.8);
plot(eeg.time, eeg.notched_data(:, channel_idx), 'Color', [0.6, 0.6, 0.6],
'LineWidth', 0.9);
```

```
plot(eeg.time, eeg.hpf_data(:, channel_idx), 'Color', [0.4, 0.4, 0.4],
'LineWidth', 1.0);
plot(eeg.time, eeg.lpf_data(:, channel_idx), 'Color', colors(1,:),
'LineWidth', 1.5);
hold off;

% Add labels and formatting
title(sprintf('Complete Filtering Chain - Channel %d: %s', channel_idx,
eeg.channels{channel_idx}), 'FontWeight', 'bold');
xlabel('Time (seconds)');
ylabel('Amplitude (µV)');
legend('Raw', 'After Notch', 'After HPF', 'After LPF', 'Location',
'northeast');
grid on;

% Add overall title
sgtitle({'Low-Pass Filtering Analysis', ...
         sprintf('Cutoff: %.1f Hz, Channel: %s', f_lpf,
eeg.channels{channel_idx})}, ...
         'FontWeight', 'bold');

% Store the final filtered data
eeg.filtered_data = eeg.lpf_data;

fprintf('Low-pass filtering and visualization complete.\n');
fprintf('EEG processing pipeline complete: Notch > High-Pass > Low-Pass\n');
fprintf('Final bandpass range: %.1f - %.1f Hz\n', f_hpf, f_lpf);
%% Step 6: Apply Bandpass Filters for EEG Bands Extraction
fprintf('Extracting EEG frequency bands using bandpass filters...\n');

% Select a representative channel for detailed analysis
channel_idx = 1;  % Using the first channel
channel_name = eeg.channels{channel_idx};
eeg_ch = eeg.filtered_data(:, channel_idx);

% Define EEG frequency bands with physiological meanings
bands = {
    'Delta', 0.5, 4,  'Deep sleep, unconsciousness';
    'Theta', 4, 8,    'Drowsiness, meditation, memory';
    'Alpha', 8, 13,   'Relaxed wakefulness, closed eyes';
    'Beta',  13, 30,  'Active thinking, focus, alertness';
    'Gamma', 30, 45,  'Cognitive processing, perception'
};

% Store band information in EEG structure
eeg.bands = bands;

% Apply bandpass filters to extract each frequency band
eeg.band_data = cell(size(bands, 1), 1);
        %size(bands, 1) gives the number of rows, i.e., how many bands.
        %cell(size(bands, 1), 1) creates a column cell array to hold the
filtered signal for each band.
fprintf('Extracting bands for channel %s:\n', channel_name);

% Create colormap for consistent coloring
colors = [
    0.0000, 0.4470, 0.7410;  % Blue for Delta
    0.8500, 0.3250, 0.0980;  % Orange for Theta
```

```
    0.9290, 0.6940, 0.1250;  % Yellow for Alpha
    0.4940, 0.1840, 0.5560;  % Purple for Beta
    0.4660, 0.6740, 0.1880   % Green for Gamma
];

% Create visualization figure
figure('Name', 'EEG Frequency Bands', 'Color', 'w', 'Position', [100, 100,
1000, 800]);

% First, plot the full filtered signal at the top
subplot(size(bands, 1) + 1, 1, 1);
plot(eeg.time, eeg_ch, 'k', 'LineWidth', 1.2);
title(sprintf('Full Filtered Signal - Channel %s', channel_name),
'FontWeight', 'bold');
ylabel('Amplitude (µV)');
grid on;
xlim([0, eeg.time(end)]);

% Apply bandpass filter for each band and plot
for i = 1:size(bands, 1)
    % Extract band parameters
    band_name = bands{i, 1};
    f_low = bands{i, 2};
    f_high = bands{i, 3};

    % Create and apply bandpass filter
    try
        [b, a] = butter(4, [f_low, f_high]/(eeg.srate/2), 'bandpass');
        filtered = filtfilt(b, a, eeg_ch);

        % Store filtered data
        eeg.band_data{i} = filtered;

        % Plot the band-filtered signal
        subplot(size(bands, 1) + 1, 1, i + 1);
        plot(eeg.time, filtered, 'Color', colors(i,:), 'LineWidth', 1.2);

        % Calculate band power
        band_power = mean(filtered.^2);

        % Add title with band information
        title(sprintf('%s Band (%.1f-%.1f Hz) - Power: %.2f µV²', ...
            band_name, f_low, f_high, band_power), 'FontWeight', 'bold',
'Color', colors(i,:));

        % Add labels and grid
        ylabel('Amp (µV)');
        grid on;
        xlim([0, eeg.time(end)]);

        % Add x-label only to the last subplot
        if i == size(bands, 1)
            xlabel('Time (seconds)');
        end

        fprintf('  - %s (%.1f-%.1f Hz): Power = %.2f µV²\n', ...
            band_name, f_low, f_high, band_power);
    catch ME
```

```
        warning('Error extracting %s band: %s', band_name, ME.message);
    end
end

% Add overall title
sgtitle({sprintf('EEG Frequency Bands - Channel %s', channel_name), ...
         sprintf('Sampling Rate: %d Hz, Recording Duration: %.2f s',
eeg.srate, eeg.time(end))}, ...
         'FontWeight', 'bold');

% Adjust spacing
set(gcf, 'PaperPositionMode', 'auto');

% Create time-frequency analysis figure
figure('Name', 'Time-Frequency Analysis', 'Color', 'w', 'Position', [150,
150, 1000, 800]);

% Create a 2x2 grid for time-frequency analysis
% 1. Spectrogram
subplot(2, 2, 1);
window_length = min(eeg.srate*2, length(eeg.time)/8);  % 2-second window or
shorter
overlap = round(window_length * 0.8);  % 80% overlap
nfft = 2^nextpow2(window_length*2);  % Next power of 2
spectrogram(eeg_ch, hamming(window_length), overlap, nfft, eeg.srate,
'yaxis');
title('Spectrogram - Full Signal', 'FontWeight', 'bold');
ylim([0, 50]);  % Limit frequency display to 50 Hz
colorbar;

% 2. Power Spectral Density
subplot(2, 2, 2);
[pxx, f] = pwelch(eeg_ch, hamming(eeg.srate*4), eeg.srate*2, [], eeg.srate);
    %[pxx, f] = pwelch(x, window, noverlap, nfft, fs);

plot(f, 10*log10(pxx), 'LineWidth', 1.5);
title('Power Spectral Density', 'FontWeight', 'bold');
xlabel('Frequency (Hz)');
ylabel('Power/Frequency (dB/Hz)');
xlim([0, 50]);  % Limit to 50 Hz
grid on;

% Add band markers to power spectrum
hold on;
colors_lighter = colors * 0.8 + 0.2;  % Make colors a bit lighter for
visibility
y_lim = ylim;
for i = 1:size(bands, 1)
    % Color the frequency band regions
    f_low = bands{i, 2};
    f_high = bands{i, 3};
    h = area([f_low f_high], [y_lim(2) y_lim(2)], y_lim(1), 'FaceColor',
colors_lighter(i,:), 'FaceAlpha', 0.3, 'EdgeColor', 'none');
    % Add text label in the middle of each band
    text((f_low + f_high)/2, y_lim(1) + 0.9*(y_lim(2)-y_lim(1)), bands{i, 1},
...
         'HorizontalAlignment', 'center', 'VerticalAlignment', 'top', ...
```

```
        'FontWeight', 'bold', 'Color', colors(i,:), 'FontSize', 8);
end
hold off;

% 3. Band Power Distribution
subplot(2, 2, 3);
band_powers = zeros(size(bands, 1), 1);
for i = 1:size(bands, 1)
    band_powers(i) = mean(eeg.band_data{i}.^2);
end

% Create the bar chart with colored bars
b = bar(1:size(bands, 1), band_powers);

% Now set the bar colors correctly
for i = 1:size(bands, 1)
    % Set the face color of each bar individually
    b.FaceColor = 'flat';  % Allow individual coloring
    b.CData = colors;      % Set the color data
end

title('Power Distribution Across Frequency Bands', 'FontWeight', 'bold');
xlabel('Frequency Band');
ylabel('Power (µV²)');
set(gca, 'XTick', 1:size(bands, 1), 'XTickLabel', {bands{:, 1}});
grid on;

% 4. Relative Band Power (as percentage)
subplot(2, 2, 4);
total_power = sum(band_powers);
rel_powers = (band_powers / total_power) * 100;

% Create pie chart with percentages
pie_handle = pie(rel_powers);

% Add custom colors to pie chart segments
num_segments = length(rel_powers);
for i = 1:num_segments
    % The odd indexed objects are the patch handles for the pie segments
    segment_index = 2*i-1; %calculates the index for each pie slice
    if segment_index <= length(pie_handle)
        %Ensures that we don't exceed the number of elements in the
pie_handle array
        set(pie_handle(segment_index), 'FaceColor', colors(i,:));%sets color
    end
end

% Create legend
legend_labels = cell(size(bands, 1), 1);
for i = 1:size(bands, 1)
    legend_labels{i} = sprintf('%s: %.1f%%', bands{i, 1}, rel_powers(i));
end
legend(legend_labels, 'Location', 'eastoutside');
title('Relative Band Power Distribution (%)', 'FontWeight', 'bold');

% Add overall title
sgtitle({sprintf('EEG Frequency Analysis - Channel %s', channel_name), ...
         'Time and Frequency Domain Analysis'}, ...
```

```
        'FontWeight', 'bold');

% Store band powers in EEG structure
eeg.band_powers = band_powers;
eeg.rel_band_powers = rel_powers;

fprintf('Frequency band extraction and analysis complete.\n');

%% Step 7: Enhanced PSD Plot for Each Frequency Band (for one channel)
figure('Name','PSD for Filtered Bands - Channel Var2','Color','w',
'Position', [100, 100, 900, 700]);

% Create a separate subplot for each band
for i = 1:5
    % Filter the signal in the current frequency band
    [b, a] = butter(4, [bands{i,2}, bands{i,3}]/(Fs/2), 'bandpass');
    filtered = filtfilt(b, a, eeg_ch);

    % Calculate power spectral density using Welch's method
    [pxx, f] = pwelch(filtered, hamming(1024), [],[], Fs);
    %[pxx, f] = pwelch(x, window, noverlap, nfft, fs);

    % Plot in the corresponding subplot
    subplot(5,1,i);
    plot(f, 10*log10(pxx), 'Color', colors(i,:), 'LineWidth', 2);

    % Add band-specific shading to highlight the frequency range
    hold on;
    y_limits = ylim;
    x = [bands{i,2}, bands{i,3}, bands{i,3}, bands{i,2}];
    y = [y_limits(1), y_limits(1), y_limits(2), y_limits(2)];
    fill(x, y, colors(i,:), 'FaceAlpha', 0.1, 'EdgeColor', 'none');

    % Add peak annotation
    [max_psd, max_idx] = max(10*log10(pxx(f >= bands{i,2} & f <=
bands{i,3})));
    f_subset = f(f >= bands{i,2} & f <= bands{i,3});
    peak_freq = f_subset(max_idx);
    text(peak_freq, max_psd, [' Peak: ' num2str(peak_freq,3) ' Hz'],
'FontWeight', 'bold');
    plot(peak_freq, max_psd, 'o', 'MarkerSize', 6, 'MarkerFaceColor',
colors(i,:), 'MarkerEdgeColor', 'k');

    % Calculate and display band power
    band_power = bandpower(filtered, Fs, [bands{i,2}, bands{i,3}]);
    text(max(xlim)*0.75, max(ylim)-5, ['Power: ' num2str(band_power, '%.2e')
' µV²'], 'FontWeight', 'bold');

    % Improve plot appearance
    xlim([0 40]);
    title([bands{i,1} ' Band (' num2str(bands{i,2}) '-' num2str(bands{i,3}) '
Hz)'], 'FontWeight', 'bold');
    ylabel('Power/Frequency (dB/Hz)');
    grid on;

    % Add minor gridlines for better readability
    grid minor;
```

```
    % Add custom frequency tick marks
    xticks(0:5:40);

    % Consistent y-axis limits for better comparison (optional, remove if
preferred to auto-scale)
    % ylim([-60 10]);

    hold off;
end

% Add common x-axis label
han = axes('visible', 'off');
han.XLabel.Visible = 'on';
xlabel(han, 'Frequency (Hz)', 'FontWeight', 'bold');

% Improve overall title
sgtitle('Power Spectral Density Analysis of EEG Frequency Bands - Channel
Var2', 'FontSize', 14, 'FontWeight', 'bold');

% Add a custom legend with band information
leg_str = {};
for i = 1:5
    leg_str{i} = [bands{i,1} ': ' num2str(bands{i,2}) '-' num2str(bands{i,3})
' Hz'];
end

% Create small axes for the legend
leg_ax = axes('Position', [0.75 0.02 0.15 0.15], 'Visible', 'off');
for i = 1:5
    line([0 0.2], [i i], 'Color', colors(i,:), 'LineWidth', 2, 'Parent',
leg_ax);
    text(0.25, i, leg_str{i}, 'Parent', leg_ax);
end
set(leg_ax, 'YLim', [0 6], 'XLim', [0 1]);

% Adjust subplot spacing for better appearance
set(gcf, 'Units', 'normalized');
set(gcf, 'Position', [0.1, 0.1, 0.8, 0.8]);
figure('Name','Magnitude and Phase Spectrum for All Bands - Channel
Var2','Color','w', 'Position', [100, 100, 1000, 800]);

for i = 1:5
    [b, a] = butter(4, [bands{i,2}, bands{i,3}]/(Fs/2), 'bandpass');
    filtered = filtfilt(b, a, eeg_ch);

    % Compute the magnitude and phase spectrum
    N = length(filtered);
    fft_signal = fft(filtered);

    % Use only the first half of the spectrum (up to Nyquist frequency)
    f = linspace(0, Fs/2, ceil(N/2));
    half_spectrum = fft_signal(1:ceil(N/2));
    mag = abs(half_spectrum);
    phase = angle(half_spectrum);

    % Normalize magnitude for better visualization
    mag = mag / max(mag);
```

```
    % Plot magnitude spectrum
    subplot(5,2,(i-1)*2+1);
    plot(f, mag, 'Color', colors(i,:), 'LineWidth', 1.5);
    title([bands{i,1} ' Band (' num2str(bands{i,2}) '-' num2str(bands{i,3}) '
Hz) - Magnitude'], 'FontWeight', 'bold');
    xlabel('Frequency (Hz)');
    ylabel('Normalized Magnitude');
    grid on;

    % Limit x-axis to a relevant frequency range (0 to upper band limit + 10
Hz)
    xlim([0, min(bands{i,3} + 10, Fs/2)]);

    % Add vertical lines to highlight band boundaries
    hold on;
    xline(bands{i,2}, '--k', 'LineWidth', 1);
    xline(bands{i,3}, '--k', 'LineWidth', 1);
    hold off;

    % Plot phase spectrum
    subplot(5,2,(i-1)*2+2);
    plot(f, phase, 'Color', colors(i,:), 'LineWidth', 1.5);
    title([bands{i,1} ' Band (' num2str(bands{i,2}) '-' num2str(bands{i,3}) '
Hz) - Phase'], 'FontWeight', 'bold');
    xlabel('Frequency (Hz)');
    ylabel('Phase (radians)');
    grid on;

    % Use same x-axis limits as magnitude plot for consistency
    xlim([0, min(bands{i,3} + 10, Fs/2)]);
    ylim([-pi, pi]);
    yticks([-pi, -pi/2, 0, pi/2, pi]);
    yticklabels({'-?', '-?/2', '0', '?/2', '?'});

    % Add vertical lines to highlight band boundaries
    hold on;
    xline(bands{i,2}, '--k', 'LineWidth', 1);
    xline(bands{i,3}, '--k', 'LineWidth', 1);
    hold off;
end

sgtitle('Magnitude and Phase Spectrum by Frequency Band - Channel Var2',
'FontSize', 14, 'FontWeight', 'bold');
for i = 1:5
    [b, a] = butter(4, [bands{i,2}, bands{i,3}]/(Fs/2), 'bandpass');
    filtered = filtfilt(b, a, eeg_ch);
    plot_mag_phase(filtered, Fs, bands{i,1}, bands{i,2}, bands{i,3},
colors(i,:), (i-1)*2+1);
end

% Adjust spacing between subplots using standard MATLAB functions
set(gcf, 'Units', 'Normalized');

% Add more space between subplots and to the figure edges
subplots = findobj(gcf, 'Type', 'axes');
for i = 1:length(subplots)
    p = get(subplots(i), 'Position');
    set(subplots(i), 'Position', p);
```

```
end

% Add some space for the suptitle
set(gcf, 'Position', [100, 100, 1000, 800]);
%% Step 8: Enhanced Real-Time EEG Visualization
% Configuration parameters
win_size = Fs * .5;       % 0.5-second window for better visualization
overlap = .8;           % 80% overlap between consecutive windows for
smoother visualization
step_size = round(win_size * (1-overlap)); % Points to move forward each step

% Use the appropriate EEG data variable
% Note: Replace eeg_data with your actual variable name if different
if exist('eeg_lpf', 'var')
    eeg_for_viz = eeg_lpf;
elseif exist('eeg_data', 'var')
    eeg_for_viz = eeg_data;
elseif exist('eeg_ch', 'var')
    eeg_for_viz = eeg_ch;
else
    error('No EEG data variable found. Please make sure to run previous steps
first.');
end

n_steps = floor((size(eeg_for_viz,1) - win_size) / step_size);
%floor used to convert from decimal to digit i.e. 2.3->2

% Create figure with adjusted position and size
h_fig = figure('Name','Enhanced Real-Time EEG Monitor - Channel Var2',
'Color','w', ...
               'Position', [100, 100, 1200, 700], 'Renderer', 'painters');

% Create main axes for EEG signal
h_ax1 = subplot(5,1,1:3);
set(h_ax1, 'NextPlot', 'replacechildren', 'XGrid', 'on', 'YGrid', 'on');
h_eeg = plot(zeros(win_size,1), 'LineWidth', 1.5, 'Color', [0 0.4470
0.7410]);
xlim([0 win_size]);
ylim([-50 50]);
title('Real-Time EEG Signal', 'FontWeight', 'bold', 'FontSize', 12);
ylabel('Amplitude (?V)');
set(h_ax1, 'XTickLabel', []);

% Create secondary axes for band powers
h_ax2 = subplot(5,1,4:5);
band_names = {'Delta', 'Theta', 'Alpha', 'Beta', 'Gamma'};
band_colors = [0 0.4470 0.7410; 0.8500 0.3250 0.0980; 0.9290 0.6940 0.1250;
              0.4940 0.1840 0.5560; 0.4660 0.6740 0.1880];
h_bar = bar(1:5, zeros(1,5), 'FaceColor', 'flat');
for b = 1:5
    h_bar.CData(b,:) = band_colors(b,:);
end
ylim([0 100]);
set(gca, 'XTickLabel', band_names, 'XGrid', 'off', 'YGrid', 'on');
title('Band Power Distribution', 'FontWeight', 'bold', 'FontSize', 12);
ylabel('Relative Power (%)');
```

```
% Create textbox for time and processing info
h_txt = annotation('textbox', [0.01, 0.01, 0.2, 0.05], 'String', '', ...
                 'FitBoxToText', 'on', 'BackgroundColor', 'w', 'EdgeColor',
'k');

% Initialize buffer for scrolling display
buffer_duration = 10;  % 10-second buffer for continuous display
buffer_size = buffer_duration * Fs;
signal_buffer = zeros(buffer_size, 1);
time_vector = (0:buffer_size-1)/Fs;

% Initialize band power history for trend display
power_history = zeros(100, 5);  % Store last 100 power measurements
trend_position = 1;

% Create status indicator
h_status = annotation('textbox', [0.80, 0.01, 0.19, 0.05], 'String',
'RECORDING', ...
                    'FitBoxToText', 'on', 'BackgroundColor', 'r',
'EdgeColor', 'k', ...
                    'FontWeight', 'bold', 'HorizontalAlignment', 'center',
'Color', 'w');

% Start simulated real-time visualization
disp('Starting enhanced real-time EEG visualization...');
tic;  % Start timer to measure actual processing time

for step = 1:n_steps
    % Update process time for more realistic simulation
    step_start_time = toc;

    % Get current segment
    idx_start = (step-1)*step_size + 1;
    idx_end = min(idx_start+win_size-1, size(eeg_for_viz,1));

    if idx_end - idx_start + 1 < win_size
        break;  % Exit if we don't have enough data
    end

    % Extract current EEG segment
    eeg_seg = eeg_for_viz(idx_start:idx_end, 1);

    % Apply bandpass filter to focus on relevant frequencies
    [b, a] = butter(4, [1 30]/(Fs/2), 'bandpass');
    filtered = filtfilt(b, a, eeg_seg);

    % Update signal buffer (shift old data and add new)
    shift_size = length(filtered);
    signal_buffer = [signal_buffer(shift_size+1:end); filtered];

    % Calculate band powers
    band_powers = zeros(1,5);
    for b = 1:5
        % Extract band boundaries
        low_freq = bands{b,2};
        high_freq = bands{b,3};

        % Apply band-specific filter
```

```
        [b_filt, a_filt] = butter(4, [low_freq high_freq]/(Fs/2),
'bandpass');
        band_filtered = filtfilt(b_filt, a_filt, filtered);

        % Calculate power as RMS
        band_powers(b) = 100 * (rms(band_filtered)^2) / (rms(filtered)^2);
    end

    % Normalize band powers for display
    if sum(band_powers) > 0
        band_powers = band_powers / sum(band_powers) * 100;
    end

    % Update power history
    power_history(trend_position,:) = band_powers;
    trend_position = mod(trend_position, size(power_history,1)) + 1;

    % Update EEG plot
    set(h_eeg, 'YData', signal_buffer, 'XData', time_vector);

    % Update band power bar chart
    set(h_bar, 'YData', band_powers);

    % Update information text
    elapsed_time = (step-1)*step_size/Fs;
    info_str = sprintf('Time: %.1f s\nWindow: %.1f s\nOverlap: %.0f%%', ...
                    elapsed_time, win_size/Fs, overlap*100);
    set(h_txt, 'String', info_str);

    % Simulate processing status changes occasionally
    if mod(step, 20) == 0
        statuses = {'RECORDING', 'ANALYZING', 'PROCESSING'};
        status_idx = mod(step/20, 3) + 1;
        set(h_status, 'String', statuses{status_idx});
    end

    % Ensure proper refresh
    drawnow;

    % Calculate how long to pause to maintain realistic timing
    step_elapsed = toc - step_start_time;
    pause_time = max(0.01, (step_size/Fs) - step_elapsed);
    pause(pause_time);
end

% Show completion
set(h_status, 'String', 'COMPLETED', 'BackgroundColor', [0.3 0.6 0.3]);
sgtitle('Real-Time EEG Monitoring Complete', 'FontSize', 14, 'FontWeight',
'bold');
```

---

# THE END